Classes

June 17, 2026 2:43 PM

https://kotlinlang.org/docs/classes.html

Classes

Edit page Last modified: 12 February 2026

> Before creating classes, consider using a data class if the purpose is to store data. Alternatively, think about extending an existing class with an extension, rather than creating a new one from scratch.

Like other object-oriented languages, Kotlin uses **classes** to encapsulate data (properties) and behavior (functions) for reusable, structured code.

Classes are blueprints or templates for objects, which you create via constructors. When you create an instance of a class, you are creating a concrete object based on that blueprint.

Kotlin offers concise syntax for declaring classes. To declare a class, use the `class` keyword followed by the class name:

```
class Person { /*...*/ }
```

The class declaration consists of:

- **Class header**, including but not limited to:
  - `class` keyword
  - Class name
  - Type parameters (if any)
  - Primary constructor (optional)
- **Class body** (optional), surrounded by curly braces `{}`, and including **class members** such as:
  - Secondary constructors
  - Initializer blocks
  - Functions
  - Properties
  - Nested and inner classes
  - Object declarations

You can keep both the class header and body to a bare minimum. If the class doesn't have a body, you can omit the curly braces `{}`:

```
// Class with primary constructor, but without a body
class Person(val name: String, var age: Int)
```

Here's an example that declares a class with a header and body, then creates an instance

```
// Person class with a primary constructor
```

Here's an example that declares a class with a header and body, then creates an instance

```
// Person class with a primary constructor
// that initializes the name property
class Person(val name: String) {
    // Class body with age property
    var age: Int = 0
}

fun main() {
    // Creates an instance of the Person class by calling the constructor
    val person = Person("Alice")

    // Accesses the instance's properties
    println(person.name)
    // Alice
    println(person.age)
    // 0
}
```

Open in Playground → Target: JVM Running on v.2.4.0

Creating instances

An instance is created when you use the class as a blueprint to build a real object to work with in your program.

To create an instance of a class, use the class name followed by parentheses `()`, similar to calling a function:

```
// Creates an instance of the Person class
val anonymousUser = Person()
```

In Kotlin, you can create instances:

- **Without arguments** (`Person()`): creates an instance using the default values, if they are declared in the class.
- **With arguments** (`Person(value)`): creates an instance by passing specific values.

You can assign the created instance to a mutable (`var`) or read-only (`val`) variable:

```
// Creates an instance using the default value
// and assigns it to a mutable variable
var anonymousUser = Person()

// Creates an instance by passing a specific value
// and assigns it to a read-only variable
val namedUser = Person("Joe")
```

It's possible to create instances wherever you need them, inside the `main()` function, within other functions, or inside another class. Additionally, you can create instances inside another function and call that function from `main()` .

It's possible to create instances wherever you need them, inside the `main()` function, within other functions, or inside another class. Additionally, you can create instances inside another function and call that function from `main()`.

The following code declares a `Person` class with a property for storing a name. It also demonstrates how to create an instance with both the default constructor's value and a specific value:

```
// Class header with a primary constructor
// that initializes name with a default value
class Person(val name: String = "Sebastian")

fun main() {
    // Creates an instance using the default constructor's value
    val anonymousUser = Person()

    // Creates an instance by passing a specific value
    val namedUser = Person("Joe")

    // Accesses the instances' name property
    println(anonymousUser.name)
    // Sebastian
    println(namedUser.name)
    // Joe
}
```

Open in Playground → Target: JVM Running on v.2.4.0

> In Kotlin, unlike other object-oriented programming languages, there is no need for the `new` keyword when creating class instances.

For information about creating instances of nested, inner, and anonymous inner classes, see the Nested classes section.

Constructors and initializer blocks

When you create a class instance, you call one of its constructors. A class in Kotlin can have a **primary constructor** and one or more **secondary constructors**.

The primary constructor is the main way to initialize a class. You declare it in the class header. A secondary constructor provides additional initialization logic. You declare it in the class body.

Both primary and secondary constructors are optional, but a class must have at least one constructor.

Primary constructor

The primary constructor sets up the initial state of an instance when it's created.

To declare a primary constructor, place it in the class header after the class name:

```
class Person constructor(name: String) { /* ... */ }
```

To declare a primary constructor, place it in the class header after the class name:

```
class Person constructor(name: String) { /*...*/ }
```

If the primary constructor doesn't have any annotations or visibility modifiers, you can omit the `constructor` keyword:

```
class Person(name: String) { /*...*/ }
```

The primary constructor can declare parameters as properties. Use the `val` keyword before the argument name to declare a read-only property and the `var` keyword for a mutable property:

```
class Person(val name: String, var age: Int) { /*...*/ }
```

These constructor parameter properties are stored as part of the instance and are accessible from outside the class.

It's also possible to declare primary constructor parameters that are not properties. These parameters don't have `val` or `var` in front of them, so they are not stored in the instance and are available only within the class body:

```
// Primary constructor parameter that is also a property
class PersonWithProperty(val name: String) {
    fun greet() {
        println("Hello, $name")
    }
}

// Primary constructor parameter only (not stored as a property)
class PersonWithAssignment(name: String) {
    // Must be assigned to a property to be usable later
    val displayName: String = name

    fun greet() {
        println("Hello, $displayName")
    }
}
```

Properties declared in the primary constructor are accessible by member functions of the class:

```
// Class with a primary constructor declaring properties
class Person(val name: String, var age: Int) {
    // Member function accessing class properties
    fun introduce(): String {
        return "Hi, I'm $name and I'm $age years old."
    }
}
```

You can also assign default values to properties in the primary constructor:

```
class Person(val name: String = "John", var age: Int = 30) { /*...*/ }
```

You can also assign default values to properties in the primary constructor:

```
class Person(val name: String = "John", var age: Int = 30) { /*...*/ }
```

If no value is passed to the constructor during instance creation, properties use their default value:

```
// Class with a primary constructor
// including default values for name and age
class Person(val name: String = "John", var age: Int = 30)

fun main() {
    // Creates an instance using default values
    val person = Person()
    println("Name: ${person.name}, Age: ${person.age}")
    // Name: John, Age: 30
}
```

Open in Playground → Target: JVM Running on v.2.4.0

You can use the primary constructor parameters to initialize additional class properties directly in the class body:

```
// Class with a primary constructor
// including default values for name and age
class Person(
    val name: String = "John",
    var age: Int = 30
) {
    // Initializes the description property
    // from the primary constructor parameters
    val description: String = "Name: $name, Age: $age"
}

fun main() {
    // Creates an instance of the Person class
    val person = Person()
    // Accesses the description property
    println(person.description)
    // Name: John, Age: 30
}
```

Open in Playground → Target: JVM Running on v.2.4.0

As with functions, you can use trailing commas in constructor declarations:

```
class Person(
    val name: String,
    val lastName: String,
    var age: Int,
) { /*...*/ }
```

Initializer blocks

The primary constructor initializes the class and sets its properties. In most cases, you can handle this with simple code.

The primary constructor initializes the class and sets its properties. In most cases, you can handle this with simple code.

If you need to perform more complex operations during instance creation, place that logic in **initializer blocks** inside the class body. These blocks run when the primary constructor executes.

Declare initializer blocks with the `init` keyword followed by curly braces `{}`. Write within the curly braces any code that you want to run during initialization:

```
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    init {
        // Initializer block runs when an instance is created
        println("Person created: $name, age $age.")
    }
}

fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
}
```

Open in Playground → Target: JVM Running on v.2.4.0

Add as many initializer blocks (`init {}`) as you need. They run in the order in which they appear in the class body, along with property initializers:

```
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    // First initializer block
    init {
        // Runs first when an instance is created
        println("Person created: $name, age $age.")
    }

    // Second initializer block
    init {
        // Runs after the first initializer block
        if (age < 18) {
            println("$name is a minor.")
        } else {
            println("$name is an adult.")
        }
    }
}

fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
    // John is an adult.
}
```

Open in Playground → Target: JVM Running on v.2.4.0

You can use primary constructor parameters in initializer blocks. For example, in the code

Open in Playground → Target: JVM Running on v.2.4.0

You can use primary constructor parameters in initializer blocks. For example, in the code above, the first and second initializers use the `name` and `age` parameters from the primary constructor.

A common use case for `init` blocks is data validation. For example, by calling the `require` function ↗:

```
class Person(val age: Int) {
    init {
        require(age > 0) { "age must be positive" }
    }
}
```

Secondary constructors

In Kotlin, secondary constructors are additional constructors that a class can have beyond its primary constructor. Secondary constructors are useful when you need multiple ways to initialize a class or for Java interoperability.

To declare a secondary constructor, use the `constructor` keyword inside the class body with the constructor parameters within parentheses `()`. Add the constructor logic within curly braces `{}`:

```
// Class header with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {

    // Secondary constructor that takes age as a
    // String and converts it to an Int
    constructor(name: String, age: String) : this(name, age.toIntOrNull() ?: 0)
        println("$name created with converted age: ${this.age}")
    }
}

fun main() {
    // Uses the secondary constructor with age as a String
    Person("Bob", "8")
    // Bob created with converted age: 8
}
```

Open in Playground → Target: JVM Running on v.2.4.0

The expression `age.toIntOrNull() ?: 0` uses the Elvis operator. For more information, see Null safety.

In the code above, the secondary constructor delegates to the primary constructor via the `this` keyword, passing `name` and the `age` value converted to an integer.

In Kotlin, secondary constructors must delegate to the primary constructor. This delegation ensures that all primary constructor initialization logic is executed before any secondary constructor logic runs.

ensures that all primary constructor initialization logic is executed before any secondary constructor logic runs.

Constructor delegation can be:

- **Direct**, where the secondary constructor calls the primary constructor immediately.
- **Indirect**, where one secondary constructor calls another, which in turn delegates to the primary constructor.

Here's an example demonstrating how direct and indirect delegation works:

```
// Class header with a primary constructor that initializes name and age
class Person(
    val name: String,
    var age: Int
) {
    // Secondary constructor with direct delegation
    // to the primary constructor
    constructor(name: String) : this(name, 0) {
        println("Person created with default age: $age and name: $name.")
    }

    // Secondary constructor with indirect delegation:
    // this("Bob") -> constructor(name: String) -> primary constructor
    constructor() : this("Bob") {
        println("New person created with default age: $age and name: $name.")
    }
}

fun main() {
    // Creates an instance based on the direct delegation
    Person("Alice")
    // Person created with default age: 0 and name: Alice.

    // Creates an instance based on the indirect delegation
    Person()
    // Person created with default age: 0 and name: Bob.
    // New person created with default age: 0 and name: Bob.
}
```

Open in Playground → Target: JVM Running on v.2.4.0

In classes with initializer blocks (`init {}`), the code within these blocks becomes part of the primary constructor. Given that secondary constructors delegate to the primary constructor first, all initializer blocks and property initializers run before the body of the secondary constructor. Even if the class has no primary constructor, the delegation still happens implicitly:

```
// Class header with no primary constructor
class Person {
    // Initializer block runs when an instance is created
    init {
        // Runs before the secondary constructor
        println("1. First initializer block runs")
    }

    // Secondary constructor that takes an integer parameter
    constructor(i: Int) {
```

```
    // Secondary constructor that takes an integer parameter
    constructor(i: Int) {
        // Runs after the initializer block
        println("2. Person $i is created")
    }
}

fun main() {
    // Creates an instance of the Person class
    Person(1)
    // 1. First initializer block runs
    // 2. Person 1 created
}
```

Open in Playground → Target: JVM Running on v.2.4.0

Classes without constructors

Classes that don't declare any constructors (primary or secondary) have an implicit primary constructor with no parameters:

```
// Class with no explicit constructors
class Person {
    // No primary or secondary constructors declared
}

fun main() {
    // Creates an instance of the Person class
    // using the implicit primary constructor
    val person = Person()
}
```

The visibility of this implicit primary constructor is public, meaning it can be accessed from anywhere. If you don't want your class to have a public constructor, declare an empty primary constructor with non-default visibility:

```
class Person private constructor() { /*...*/ }
```

On the JVM, if all primary constructor parameters have default values, the compiler implicitly provides a parameterless constructor that uses those default values.

This makes it easier to use Kotlin with libraries such as Jackson ↗ or Spring Data JPA ↗, which create class instances through parameterless constructors.

In the following example, Kotlin implicitly provides a parameterless constructor `Person()` that uses the default value `""`:

```
class Person(val personName: String = "")
```

Inheritance

Inheritance

Class inheritance in Kotlin allows you to create a new class (derived class) from an existing class (base class), inheriting its properties and functions while adding or modifying behavior.

For detailed information about inheritance hierarchies and how to use of the `open` keyword, see the Inheritance section.

Abstract classes

In Kotlin, abstract classes are classes that can't be instantiated directly. They are designed to be inherited by other classes which define their actual behavior. This behavior is called an **implementation**.

An abstract class can declare abstract properties and functions, which must be implemented by subclasses.

Abstract classes can also have constructors. These constructors initialize class properties and enforce required parameters for subclasses. Declare an abstract class using the `abstract` keyword:

```
abstract class Person(val name: String, val age: Int)
```

An abstract class can have both abstract and non-abstract members (properties and functions). To declare a member as abstract, you must use the `abstract` keyword explicitly.

You don't need to annotate abstract classes or functions with the `open` keyword because they are implicitly inheritable by default. For more details about the `open` keyword, see Inheritance.

Abstract members don't have an implementation in the abstract class. You define the implementation in a subclass or inheriting class with an `override` function or property:

```
// Abstract class with a primary constructor that declares name and age
abstract class Person(
    val name: String,
    val age: Int
) {
    // Abstract member
    // Doesn't provide implementation,
    // and it must be implemented by subclasses
    abstract fun introduce()

    // Non-abstract member (has an implementation)
    fun greet() {
        println("Hello, my name is $name.")
    }
}

// Subclass that provides an implementation for the abstract member
class Student(
    name: String,
```

```kotlin
// Subclass that provides an implementation for the abstract member
class Student(
    name: String,
    age: Int,
    val school: String
) : Person(name, age) {
    override fun introduce() {
        println("I am $name, $age years old, and I study at $school.")
    }
}

fun main() {
    // Creates an instance of the Student class
    val student = Student("Alice", 20, "Engineering University")

    // Calls the non-abstract member
    student.greet()
    // Hello, my name is Alice.

    // Calls the overridden abstract member
    student.introduce()
    // I am Alice, 20 years old, and I study at Engineering University.
}
```

Open in Playground → Target: JVM Running on v.2.4.0

Companion objects

In Kotlin, each class can have a companion object. Companion objects are a type of object declaration that allows you to access its members using the class name without creating a class instance.

Suppose you need to write a function that can be called without creating an instance of a class, but it is still logically connected to the class (such as a factory function). In that case, you can declare it inside a companion object declaration within the class:

```kotlin
// Class with a primary constructor that declares the name property
class Person(
    val name: String
) {
    // Class body with a companion object
    companion object {
        fun createAnonymous() = Person("Anonymous")
    }
}

fun main() {
    // Calls the function without creating an instance of the class
    val anonymous = Person.createAnonymous()
    println(anonymous.name)
    // Anonymous
}
```

Open in Playground → Target: JVM Running on v.2.4.0

Open in Playground → Target: JVM Running on v.2.4.0

If you declare a companion object inside your class, you can access its members using only the class name as a qualifier.

For more information, see Companion objects.